

MICROPROCESORUL 8086 (I)

1. OBIECTUL LUCRĂRII

Lucrarea urmărește studierea microprocesorului 8086, atât din punct de vedere hardware cât și software, precum și realizarea de aplicații cu acest microprocesor. Este prezentat programul Emul 8086 precum și operațiile de bază cu acesta pentru crearea și simularea programelor în limbaj de asamblare.

2. BREVIAR TEORETIC

Microprocesorul 8086 este un procesor pe 16 biți, având magistrala de adrese de 20 linii, prin intermediul căreia se poate adresa 1 Moctet de memorie. Este realizat practic într-o capsulă cu 40 pini, fiind alimentat de la o singură sursă de + 5 V (2 pini de masă).

Magistralele de date și adrese sunt multiplexate. La începutul unui ciclu mașină, pe aceste linii se transmite informația de adresă. Emisia adresei este însoțită de generarea unui semnal de comandă, care se numește ALE - address latch enable (validarea latch-urilor de adrese). Folosind acest semnal, informația de adresă este memorată în latch-uri externe, care păstrează astfel această informație, pe toată durata ciclului mașină. În cea de-a doua parte, pe liniile acestei magistrale se transmit informațiile de date. Sensul de transmitere este dat de tipul ciclului de lucru (citire sau scriere).

Procesorul poate lucra în două moduri:

- a) minimal
- b) maximal

Modul minimal este folosit în sisteme cu complexitate redusă având un singur microprocesor.

Modul maximal este folosit în sisteme complexe în care există mai multe microprocesoare, eventual coprocesoare.

Spațiul adreselor pentru circuitele de intrare-ieșire este de 64 Kocteți.

Microprocesorul 8086 nu formează singur unitate centrală (cum este cazul $\mu P Z-80$). Pentru acest lucru mai sunt necesare un circuit 8284 (generator de ceas), iar în modul maximal circuitul 8288 (controler de sistem).

De asemenea, în ambele moduri de lucru sunt necesare circuite adiționale pentru formarea magistralelor de adrese și date. Este vorba de circuitul 8282 (8 latch-uri pe capsulă) - pentru magistrala de adrese - și, respectiv, 8286 (8 bufferi bidirecționali pe capsulă) - pentru magistrala de date.

Îndrumar de laborator 6

Microprocesorul 8086 dispune de mecanisme pentru primirea și tratarea întreruperilor hardware nemascabile și mascabile, precum și pentru întreruperi software.

În setul de instrucțiuni există operațiile de înmulțire și împărțire pe 16 biți, cu și fără semn.

Are o singură intrare de ceas pe care trebuie să se aplice un semnal oscilatoriu digital, având factorul de umplere 1/3 și nivelul de "1" -logic de minim 3,5 V. Frecvența maximă a semnalului de ceas pentru prima versiune (HMOS - high speed MOS) a fost de 5 MHz, dar există și variante ce lucrează la 20 MHz (CMOS).

În modul minimal, semnalele magistralei de control sunt generate direct de către procesor, iar în modul maximal aceste semnale sunt emise de circuitul 8288.

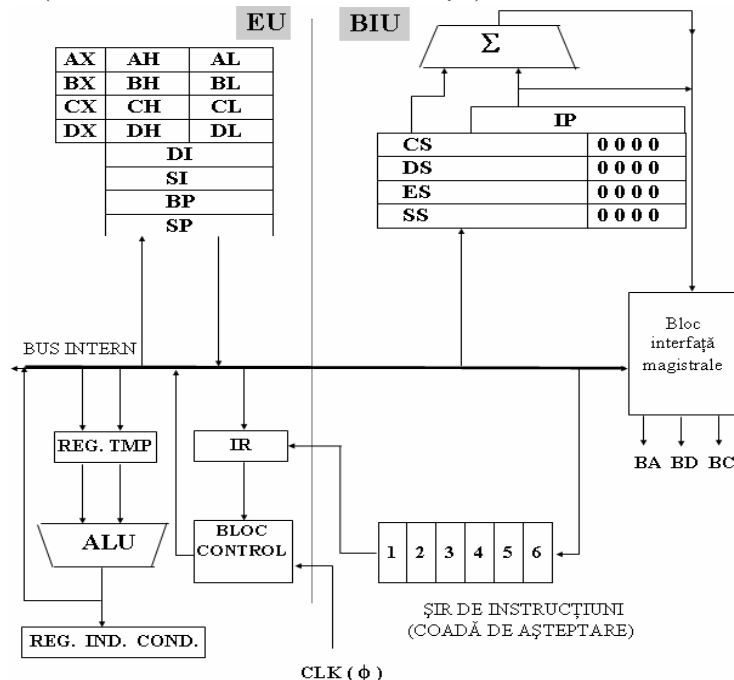
Firma INTEL a produs după μP 8086 următoarele microprocesoare : 80286, 80386, 80486, P5 (Pentium), etc.

Ele au fost create păstrând compatibilitatea cu variantele anterioare din punct de vedere software.

Structura funcțională a μP 8086

Microprocesorul 8086 este constituit din două părți :

- BIU (bus interface unit - unitatea de interfațare cu magistralele)
- EU (execution unit - unitatea de execuție)

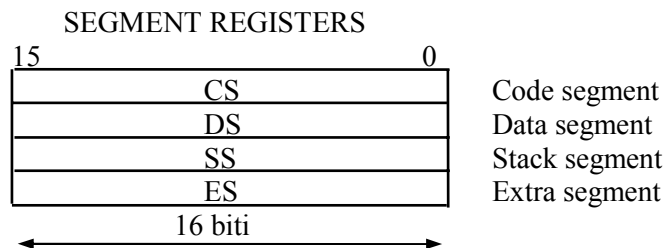
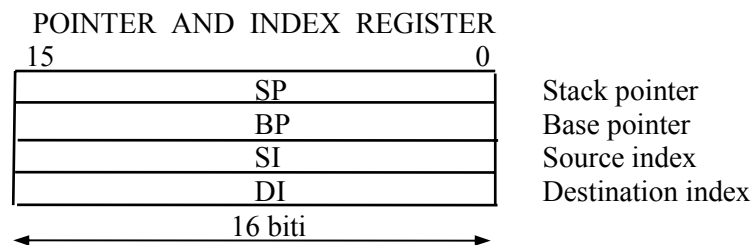
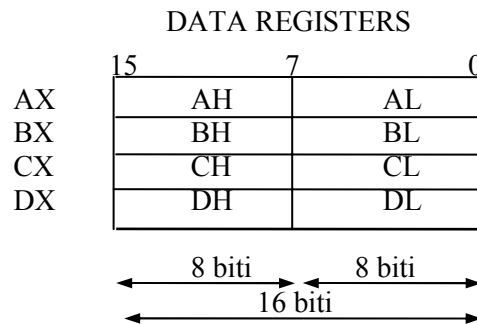


Îndrumar de laborator 6

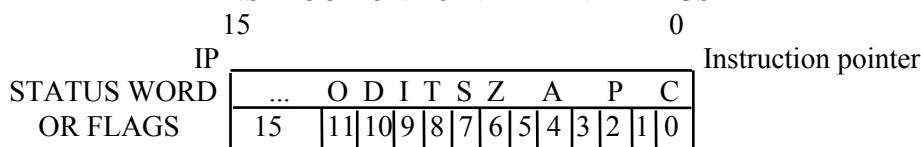
BIU - realizează interfațarea microprocesorului cu exteriorul prin intermediul unei magistrale de adrese de 20 de linii, din care primele 16 sunt multiplexate cu magistrala de date, iar ultimele 4 sunt multiplexate cu informații de stare. De asemenea, generează semnalele de comandă pentru citirea/scrierea operanzilor din/în memorie și pentru efectuarea operațiilor de intrare/ieșire. Realizează aducerea în avans a instrucțiunilor în blocul numit ȘIR DE INSTRUCȚIUNI (Queue Instruction).

EU - decodifica și execută instrucțiunile pe care le extrage succesiv din șirul de instrucțiuni al BIU.

Regiștrii microprocesorului 8086 sunt prezentați în continuare cu funcțiile lor:



INSTRUCTION POINTER AND FLAGS



REGISTRU	FUNCTIE
AX	Înmulțire, împărțire, I/O pe cuvânt
AL	Înmulțire, împărțire, I/O, pe octet transfer, aritmetica zecimală
AH	Înmulțire, împărțire pe octet
BP	Folosit la adresări de tip bazat
BX	Transfer
CX	Operații cu șiruri
CL	Contor pentru deplasări și rotații
DX	Înmulțire, scădere pe cuvânt, I/O indirect
SP	Stiva operații
SI	Operații cu șiruri
DI	Operații cu șiruri

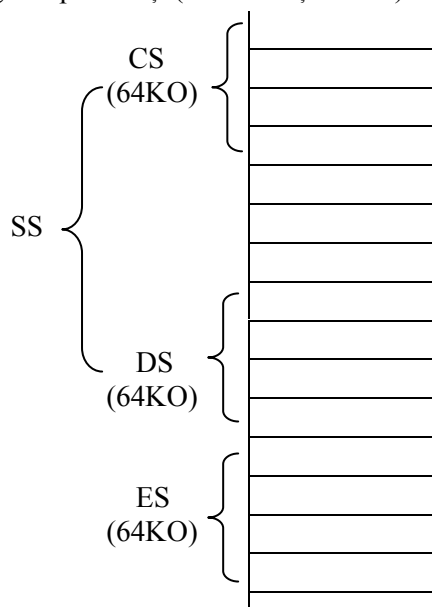
Microprocesorul 8086 folosește un mecanism de adresare segmentată a memoriei, adresa fizică a unei locații de memorie obținându-se prin efectuarea unei sume între doi operanzi, unul dintre aceștia fiind conținutul unui registru specializat, denumit segment, deplasat stânga cu patru biți (sau înmulțit cu 16).

Sunt 4 astfel de regiștrii segment:

- CS - code segment
- DS - data segment
- ES - extra segment
- SS - stack segment

Fiecare registru segment poate controla o zonă de 64 Kocteti din memoria externă adresată (ce poate fi de 1Moctet). În felul acesta, direct, μP -ul poate să aibă acces la $4 \times 64 \text{ Kocteti} = 256 \text{ Kocteti}$ din spațiul de memorie de 1Moctet ce poate fi adresat.

Zonele controlate de registrul segment pot fi distincte, pot fi unele în continuarea celorlalte (overlapping) sau pot fi suprapuse total sau parțial.



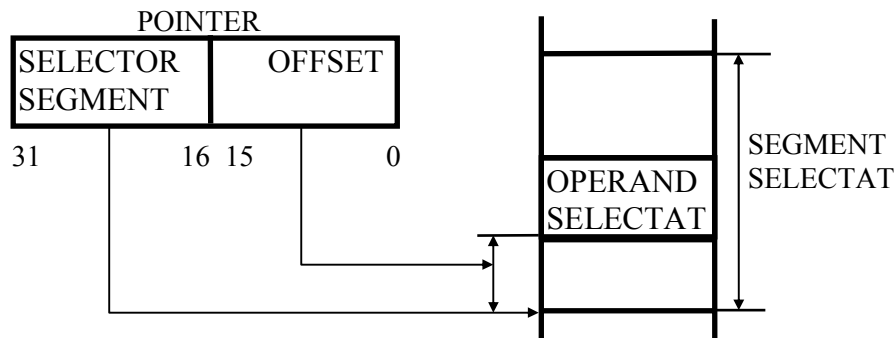
Îndrumar de laborator 6

Fiecare locație de memorie are asociate două adrese: adresa logică și adresa fizică (efectivă).

Adresa logică este compusă din două elemente:

- valoarea bazei segmentului selectat, conținută într-un registru segment;
- deplasament (offset) în cadrul segmentului.

Adresa logică este adresa cu care operează programele utilizatorului.



Adresa fizică (efectivă) se calculează în funcție de modul de adresare al datelor prezent în instrucțiune. În caz general se utilizează următoarea formă de calcul:

$$\text{adresa efectivă} = (\text{BX sau BP}) + (\text{SI sau DI}) + (\text{depl. pe 8 sau 16 biti})$$

Adresa fizică a unei locații de memorie cu care μP lucrează se obține în urma următoarei operații :

Registrul
segment
implicat

19 20 biti 0

X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	X	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

+

Adresa
logica în
cadrul
segmentul
ui

15	16 biti															0
Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y

Adresa
fizică a
locației de
memorie
căutate

Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Z	Y	Y	Y	Y
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

Îndrumar de laborator 6

Obținerea adresei fizice este realizată de către sumatorul existent în BIU. Registrul cod segment gestionează de fapt spațiul de memorie în care se află instrucțiunile propriu-zise pe care microprocesorul le execută. El este folosit în formarea adreselor fizice ale instrucțiunilor ce urmează a fi extrase din memorie și executate (pentru ciclurile fetch). În această situație, adresa logică din cadrul segmentului este reprezentată de conținutul registrului IP (instruction pointer) - indicatorul de instrucțiuni. Acest registru are un rol similar cu cel al PC-ului de la microprocesorul Z-80. Toți cei patru regiștri segment sunt regiștri pe 16 biți. La calculul adresei fizice a unei locații de memorie, acești regiștri sunt deplasați la stânga cu 4 poziții, pe pozițiile c.m.p.s. introducându-se zerouri. Cele 4 zerouri înseamnă o multiplicare cu 16. Registrul DS este folosit pentru adresarea datelor. Adresa logică din cadrul acestui segment poate fi construită dintr-un număr pe 16 biți sau din conținutul unui alt registru al μP (dar nu IP) sau din suma conținutului unor regiștri și a unor constante. Registrul ES este folosit tot pentru gestionarea datelor, dar în particular a șirurilor de date. Registrul SS se ocupă de gestionarea stivei care se creează în memorie. Pentru registrul SS, adresa logică în cadrul segmentului este de obicei constituită din conținutul regiștrilor SP și BP. Regiștrii segment pot fi încărcăși prin instrucțiuni software, în felul acesta putându-se acoperi întreaga zonă de memorie în pași succesivi.

Instrucțiunile software se împart în 10 grupe, după rolul lor:

- instrucțiuni pentru transferul informațiilor,
- instrucțiuni aritmetice,
- instrucțiuni de salt,
- instrucțiuni pentru lucrul cu subrutinele,
- instrucțiuni de intrare/ieșire,
- instrucțiuni pentru întreruperi,
- instrucțiuni pentru iterații,
- instrucțiuni pentru lucrul cu șiruri,
- instrucțiuni de control.

În urma execuției unor instrucțiuni sunt afectați indicatorii de condiții ai microprocesorului. Indicatorii se află în registrul indicatorilor de condiții. Acesta este un registru pe 16 biți, din care doar 9 sunt semnificativi. Forma registrului este următoarea:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
X	X	X	X	O	D	IF	TF	SF	ZF	X	A	X	PF	X	C
				F	F						F				F

Indicatorii se poziționează în urma execuției de către microprocesor a unor operații; se pot testa și se pot lua decizii în funcție de valoarea acestora.

Indicatorii de condiții ai microprocesorului 8086 sunt:

Îndrumar de laborator 6

- CF (Carry) = indicatorul de transport; se activează atunci când apare un transport sau un împrumut din/in rangul cel mai semnificativ al rezultatului;
- PF (Parity) = indicator de paritate; se activează atunci când rezultatul unei operații aritmetice sau logice produce un octet cu un număr par de biți aflați în starea "1"- logic;
- AF (Auxiliary Carry) = indicator de transport auxiliar; se activează când apare un transport sau un împrumut la operațiile aritmetice între biții D3 și D4 ai rezultatului;
- ZF (Zero) = indicator de zero; se activează dacă rezultatul unei operații aritmetice sau logice este zero;
- SF (Sign) = indicator de semn; conține bitul cel mai semnificativ al rezultatului unei operații aritmetice efectuate între doi operanzi exprimați în cod complement față de doi; bitul va fi "1" pentru rezultat negativ și "0" pentru un rezultat pozitiv;
- OF (Overflow) = indicator de depășire; se activează atunci când apare depășirea capacității de reprezentare a rezultatului la operații efectuate cu operanzi exprimați în cod complement față de doi; bitul va fi "1" atunci când apare o depășire;
- TF (Trap) = indicatorul folosit la depanarea programelor, permițând rularea acestora pas cu pas; dacă bitul are valoarea "1", microprocesorul va lucra în modul instrucțiune cu instrucțiune;
- IF (Interrupt) = indicator pentru mecanismul de întreruperi mascabile; dacă bitul are valoarea "1", microprocesorul accepta cererile de întrerupere externe, primite la intrarea INTR altfel, acest mecanism este dezactivat;
- DF (Direction) = indicator pentru direcție; se folosește la operațiile cu șiruri, indicând sensul de lucru cu acestea; dacă este "1" instrucțiunea incrementează adresa operandului din șir după fiecare transfer, iar dacă este "0" adresa operandului este decrementată.

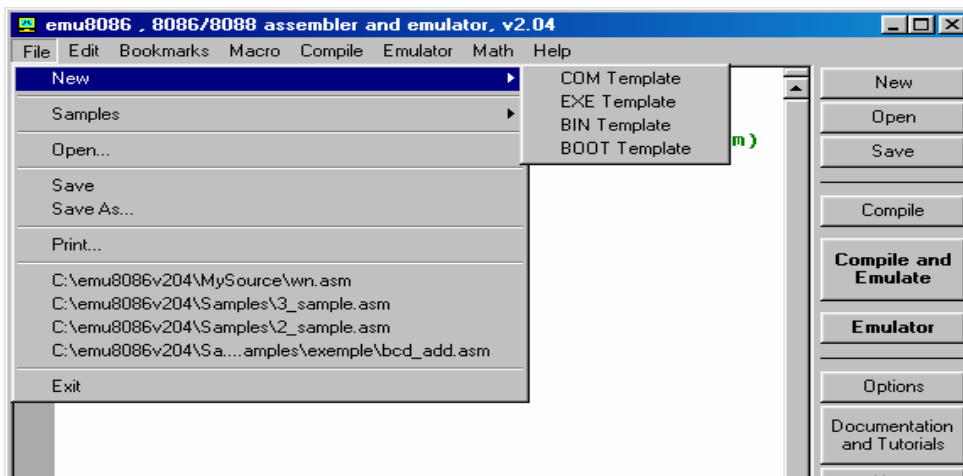
Există instrucțiuni pentru poziționarea indicatorilor DF, IF, TF și CF.

PREZENTAREA APLICAȚIEI EMU8086 ASAMBLOR, EMULATOR 8086

Pachetul emu8086 este compus dintr-un editor de surse 8086, asamblor, dezasamblor și un emulator software (Virtual PC) cu debugger.

Pentru a crea un nou fișier sursă (asm) se selectează din meniul principal:
File -> New.

Îndrumar de laborator 6



În meniu vor fi disponibile mai multe opțiuni. În funcție de opțiunea selectată, va fi generat un schelet de program în care sunt introduse directivele de compilare necesare pentru generarea fișierelor COM, EXE, BIN sau crearea unei secvențe pentru sectorul de BOOT. Un fișier nou poate fi creat și prin apăsarea butonului **New** din setul de butoane aflate în dreapta ferestrei principale.

În fereastra principală poate fi introdus, în acest moment, programul.

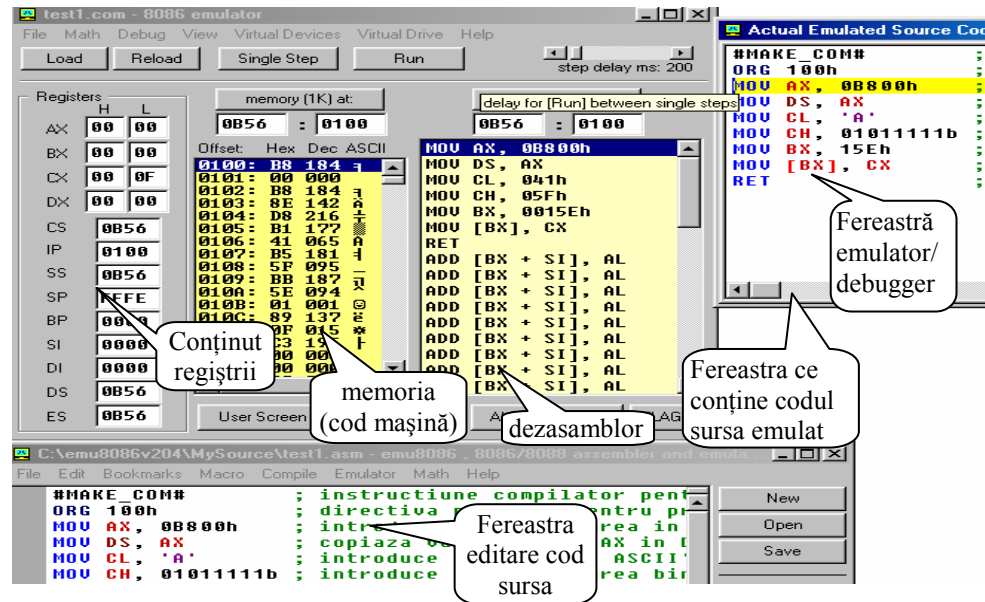
De exemplu, introduceți următorul program:

```
#MAKE_COM#           ; instructiune compilator pentru a crea un fisier COM.
ORG 100h              ; directiva necesara pentru programele COM.
MOV AX, 0B800h        ; introduce in AX valoarea in hexazecimal B800h.
MOV DS, AX            ; copiaza valoarea din AX in DS.
MOV CL, 'A'           ; introduce in CL codul ASCII 'A' (41h).
MOV CH, 01011111b     ; introduce in CH valoarea binara 01011111 (5fh).
MOV BX, 15Eh          ; introduce in BX 15Eh.
MOV [BX], CX          ; copiaza continutul lui CX in memorie la adresa
B800:015E.
RET                  ; revine in sistemul de operare.
```

Pentru a compila programul și a intra în modul emulator se apasă butonul **Compile and Emulate** din setul de butoane aflate în dreapta ferestrei sau se selectează din meniul principal: **Compile -> Compile and Load in Emulator**.

Programul este compilat și dacă nu există erori, sunt generate fișierele în funcție de tipul programului ales (com, exe, bin sau boot) apoi se poate urmări funcționarea programului editat folosind emulatorul.

Îndrumar de laborator 6



În fereastra emulator/debugger poate fi vizualizat conținutul regiștrilor, conținutul memoriei (codul mașină al instrucțiunilor și eventualele date) precum și programul dezasamblat (codul sursa obținut din codul mașină).

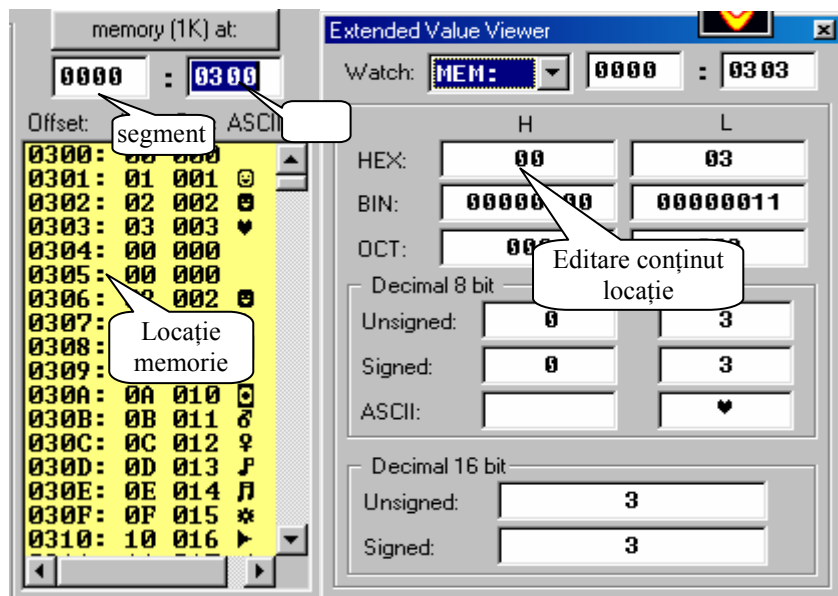


Dacă se apasă butonul **Run** din partea de sus a ferestrei emulator/debugger emulatorul va executa programul, cu întârzieri mari, pentru a se putea vedea modul în care sunt parcurse instrucțiunile. De asemenea este prezentă și opțiunea de rulare pas cu pas a programului, la apăsarea pe butonul **Single Step** va fi executată numai o instrucțiune. Resetarea și reluarea execuției programului de la început se face prin apăsarea butonului **Reload**. Încărcarea unui alt fișier de tip .bin, .com, .exe sau boot pentru dezasamblare și depanare se face prin apăsare buton **Load**.



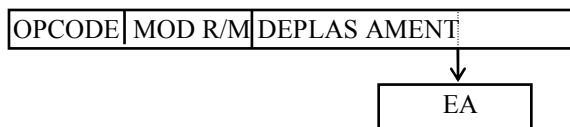
Prin apăsarea butonului **User Screen** din partea de jos a ferestrei emulator este simulat ecranul aplicației, folosit pentru programe care utilizează funcții video sau lucrează cu memoria video (cum este cazul exemplului de mai sus). Mai pot fi vizualizate unitatea aritmetică și logică, conținutul stivei și starea indicatorilor.

Pentru modificarea manuala a conținutului memoriei (inițializarea unor locații) se va introduce adresa dorita (segment și offset) în câmpul memorie apoi se apasă butonul **memory at** pentru afișarea zonei respective. Se poziționează pe adresa respectivă și se dă dublu clic pe linia respectivă. În fereastra care apare se permite editarea valorii de la locația respectivă.

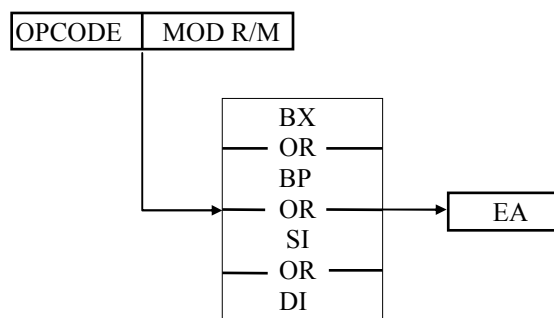


MODURI DE ADRESARE ALE MEMORIEI

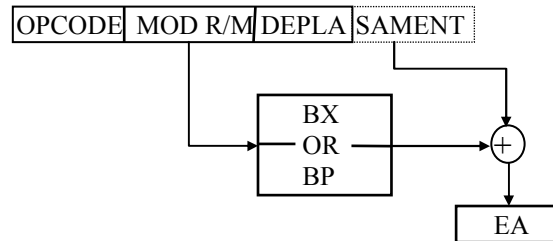
1. ADRESARE DIRECTĂ:



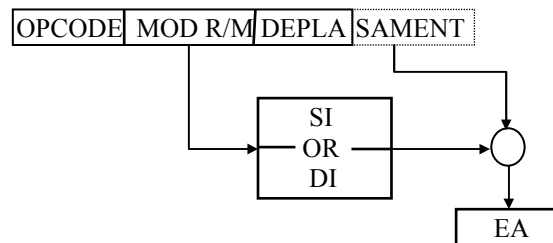
2. ADRESARE INDIRECTĂ
PRIN REGISTRU:



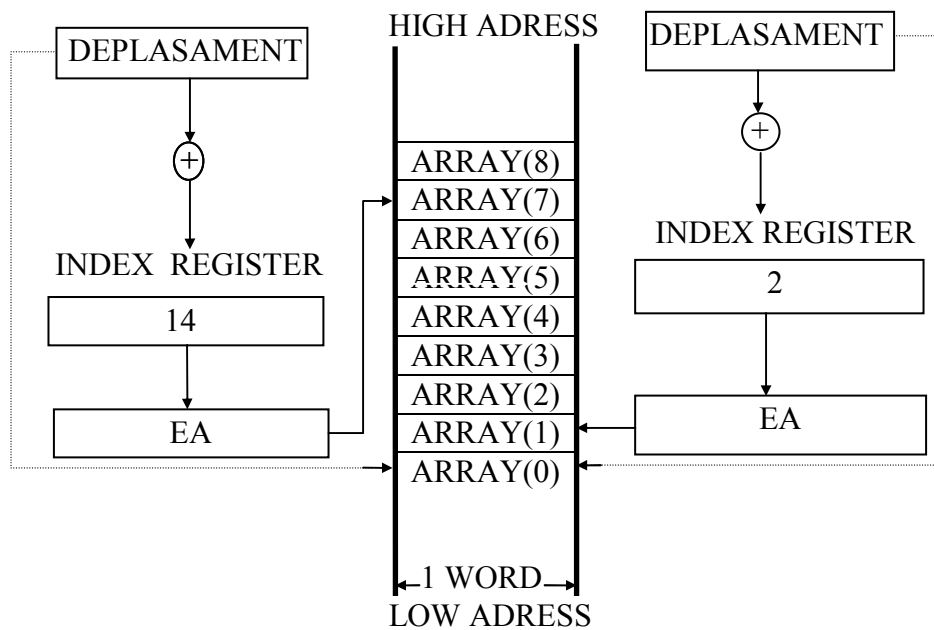
3.ADRESARE CU BAZĂ:



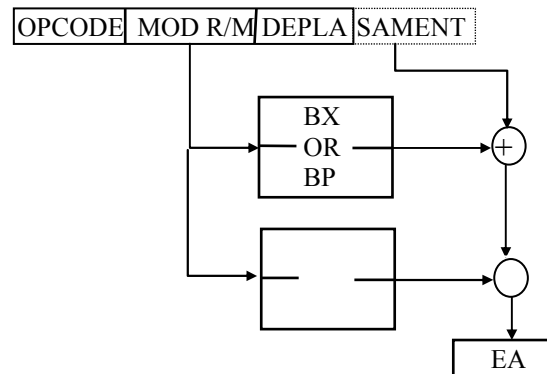
4.ADRESARE CU INDEX:



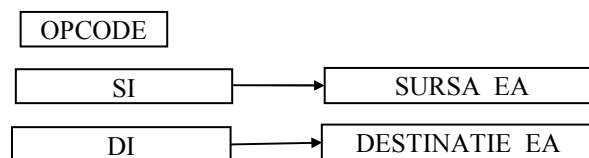
EXEMPLU DE ADRESARE CU INDEX:



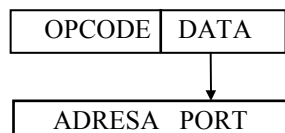
5. ADRESARE CU BAZĂ ȘI INDEX:



7. ADRESARE OPERAND ȘIR:



8. ADRESARE PORT I/O:



ADRESARE DIRECTĂ PORT



ADRESARE INDIRECTĂ PORT

INSTRUCȚIUNI PENTRU TRANSFERUL DATELOR

Instrucțiunile din această grupă sunt folosite pentru transferul informațiilor între regiștri sau între regiștri și memorie (locații de memorie).

- MOV D , S (move) D = destinație
S = sursă

Îndrumar de laborator 6

DESTINAȚIE	SURSĂ
memorie	acumulator
acumulator	memorie
registru	registru
memorie	registru
registru	imediat
memorie	imediat
registru	memorie
registru segment	registru 16 biți
registru segment	memorie 16 biți
registru 16 biți	registru segment
memorie 16 biți	registru segment

Instrucțiunea transferă conținutul sursei în destinație. Sunt permise mai multe combinații S - D. Transferul se poate efectua între doi regiștri sau între un registru și memorie, neputându-se executa transferul memorie-memorie. Se pot transfera date la nivel de octet sau la nivel de cuvânt.

Ex: MOV AL , 0ACH - transferă octetul 0ACH în registrul AL,
 MOV AX , DX - transferă conținutul registrului DX în registrul AX,
 MOV AX , 1200H - în AL se transferă 00H iar în AH se transferă 12H,
 MOV AX , [1200H]- presupunem (DS)=2000H; instrucțiunea transferă conținutul locației de memorie cu adresa 21200H în AL și conținutul locației de memorie cu adresa 21201H în AH. Adresa 21200 s-a obținut folosind schema de adresare prin registrul segment prezentată anterior.

2. XCHG D , S (Exchange)

DESTINATIE	SURSĂ
acumulator (16 biți)	registru 16 biți
Memorie	Registru
Registru	Registru

Instrucțiunea schimbă conținutul registrilor sursa și destinație.

3. PUSH S (push)

Instrucțiunea transferă cuvântul sursă în stiva. S poate fi registru 16 biți, registru segment (dar nu și CS), locație de memorie.

4. PUSHF

Instrucțiunea transferă în stivă registrul indicatorilor de condiții.

5. POP D (pop)

Instrucțiunea extrage un cuvânt din stivă și-l depune la destinație. D poate fi registru 16 biți, registru segment (dar nu și CS), locație de memorie.

Îndrumar de laborator 6

6. POPF

Instrucțiunea extrage din stivă registrul indicatorilor de condiții.

7. LAHF (Load AH from Flags)

Instrucțiunea transferă în AH registrul indicatorilor de condiții în formatul:

7	6	5	4	3	2	1	0
SF	ZF	X	AF	X	PF	X	CF

8. SAHF (Store AH to Flags)

Instrucțiunea transferă în registrul indicatorilor de condiții, conținutul registrului AH afectând doar biții: SF, ZF, AF, PF, CF.

9. LEA Reg. 16, DEPL (Load Effective Address)

Instrucțiunea are ca operand o adresă și nu o dată. În urma execuției ei se transferă într-un registru pe 16 biți, valoarea DEPL ce reprezintă deplasamentul adresei unui operand în raport cu adresa de început a segmentului respectiv.

10. LDS Reg. 16, DEPL (Load Pointer Using DS)

Instrucțiunea este asemănătoare cu LEA, doar că transferă un operand pe 32 biți în registrul specificat și în registrul DS. Deplasamentul adresei la care se găsește primul cuvânt al operandului este DEPL și acest cuvânt este transferat în unul din registre. Conținutul următoarelor două locații de memorie constituie al doilea cuvânt al operandului și se transferă în registrul DS.

11. LES Reg. 16, DEPL (Load Pointer Using ES)

Instrucțiunea lucrează în mod similar ca și LDS, doar ca registrul de segment implicat în transfer nu este DS ci ES.

12. XLAT (Translate)

Operatie efectuada: $AL \leftarrow (BX + AL)$

Instrucțiunea este utilizată în cazul operanzilor de căutare într-o tabelă. Tabela se găsește în segmentul de date, registrul BX conține adresa de început a tabelii, iar conținutul registrului AL este deplasamentul adresei la care se găsește operandul, în raport cu adresa de început a tabelii. Operandul găsit la adresa calculată astfel înlocuiește conținutul registrului AL. Deci adresa fizică a operandului se obține după relația: $AF = DS + BX + AL$.

INSTRUCȚIUNI ARITMETICE

Microprocesorul 8086 dispune de instrucțiuni pentru operații de adunare, scădere, înmulțire și împărțire asupra unor operanzi la nivel de octet și cuvânt, cu și fără semn și la nivel de nibble (jumătate de octeți compactați, adică doi digiți pe octet sau necompactați, adică un digit pe octet).

INSTRUCȚIUNI PENTRU ADUNARE

1. **ADD D , S (Add)** D = destinația; S = sursa;
Operația efectuată este: $D \leftarrow D + S$

DESTINATIE	SURSA
registru	registru
registru	memorie
memorie	registru
registru	imediat
memorie	imediat
acumulator	imediat

Indicatorii afectați în urma execuției instrucțiunii sunt:

OF, SF, ZF, AF, PF, CF.

2. **ADC D,S (Adc)**

DESTINATIE	SURSA
registru	registru
registru	memorie
memorie	registru
registru	imediat
memorie	imediat
acumulator	imediat

Operația efectuată este:

$D \leftarrow D + S + CF$.

Indicatorii afectați în urma execuției instrucțiunii sunt:

OF, SF, ZF, AF, PF, CF.

3. **INC D (Increment)** D poate fi: - registru pe 16 biti;
- registru pe 8 biti;
- locație de memorie.

Operație efectuată: $D \leftarrow D + 1$;

Indicatorii afectați în urma execuției instrucțiunii sunt: OF, SF, ZF, AF, PF.

4. **AAA (ASCII Adjust for Addition)**

Instrucțiunea este utilizată pentru corectarea rezultatului obținut prin adunarea a doi octeți reprezentați sub forma de digiți necompacți. Dacă operanzii sunt în cod ASCII rezultatul va avea jumătatea superioară zero.

Indicatori afectați: AF, CF.

5. **DAA (Decimal Adjust for Addition)**

Instrucțiunea urmează unei instrucțiuni de adunare a doi operanzi reprezentați sub forma de digiți compacți, având rolul de a corecta rezultatul obținut.

Indicatori afectați: OF, SF, ZF, AF, PF, CF.

INSTRUCȚIUNI DE SCADERE

6. SUB D, S (Subtract)

DESTINATIE	SURSA
Registru	registru
registru	memorie
memorie	registru
acumulator	imediat
registru	imediat
memorie	imediat

Operația efectuată este:
 $D \leftarrow D - S$

Indicatori afectați:
 OF, SF, ZF, AF, PF, CF.

7. SBB D, S (Subtract with Borrow)

DESTINATIE	SURSA
registru	registru
registru	memorie
memorie	registru
acumulator	imediat
registru	imediat
memorie	imediat

Operația efectuată este:
 $D \leftarrow D - S - CF$

Indicatori afectați:
 OF, SF, ZF, AF, PF, CF.

8. DEC D (Decrement)

Operație efectuată: $D \leftarrow D - 1$; D poate fi:

- registru pe 16 biți;
- registru pe 8 biți;
- locație de memorie.

Indicatori afectați: OF, SF, ZF, AF, PF, CF.

9. NEG D (Negate)

Operația efectuată: $D \leftarrow 0 - D$; D poate fi:

- registru;
- locație de memorie.

Indicatori afectați: OF, SF, ZF, AF, PF, CF $\leftarrow 1$.

10. CMP D, S (Compare)

Operație efectuată: $D - S$;

- poziționare indicatori;
- rezultatul operației nu se depune nicăieri.

Indicatori afectați: OF, SF, ZF, AF, PF, CF.

D	S
Registru	registru
registru	memorie
Memorie	registru
Registru	imediat
Memorie	imediat
Acumulator	imediat

Îndrumar de laborator 6

11. AAS (ASCII Adjust for Subtraction)

Instrucțiunea corectează rezultatul obținut în urma scăderii a doi operanzi prezentați sub forma de digiți necompactați. Dacă operanzii reprezintă coduri ASCII, jumătatea superioară a registrului este zero.

Operații efectuate: if (AL & 0FH) > 9 or AF=1 then

AL ← AL - 6

AH ← AH - 1

AF ← 1

CF ← AF

AL ← AL & 0FH

Indicatori afectați:
AF, CF.

12. DAS (Decimal Adjust for Subtraction)

Instrucțiunea corectează rezultatul obținut în urma scăderii a doi operanzi reprezentați sub forma de digiți compactați. DAS și AAS trebuie să urmeze instrucțiunilor de scădere.

Operații efectuate : if (AL & 0FH) > 9 or AF=1 THEN

AL ← AL - 6

AF ← 1

if AL > 9FH or CF=1 then

AL ← AL - 60H

CF ← 1

Indicatori afectați:
OF, SF, ZF,
AF, PF, CF.

INSTRUCȚIUNI DE ÎNMULȚIRE

13. MUL S (Multiply)

Operații efectuate: AX ← AL x S8 Indicatori afectați: OF, CF.

DX, AX ← AX x S16

Sursa poate fi: - registru pe 8 biți;

- conținutul unei locații de memorie pe 8 biți;

- registru pe 16 biți;

- conținutul unei locații de memorie pe 16 biți.

14. IMUL S (Integer Multiply)

Instrucțiunea acționează ca și MUL S cu deosebirea ca operanzii sunt considerați cu semn.

15. AAM (ASCII Adjust for Multiply)

Instrucțiunea corectează rezultatul obținut prin înmulțirea a doi operanzi reprezentați sub forma de digiți necompactați.

Indicatori afectați: PF, ZF, SF.

INSTRUCȚIUNI DE ÎMPĂRȚIRE

16. DIV S (Divide)

Instrucțiunea împarte conținutul acumulatorului la conținutul sursei.

Operanzii sunt considerați fără semn. Dacă câtul depășește capacitatea regiștrilor, adică este 0FFH, respectiv 0FFFFH se generează o întrerupere de tip zero.

OPERATII	OPERANZI	
AL ← Q(AX / S8)	} registru pe 8 biți;	Q = câtul împărțirii R = restul împărțirii
AH ← R(AX / S8)		
AX ← Q(DX,AX / S16)	} registru pe 16 biți;	
DX ← R(DX,AX / S16)		

17. IDIV (Integer Divide)

Instrucțiunea acționează ca și instrucțiunea DIV, cu mențiunea ca operanzii sunt considerați cu semn. În acest caz depășirea capacității regiștrilor va însemna pentru cât valoarea 7FH dacă împărțitorul este un octet, respectiv 7FFFH dacă împărțitorul este un cuvânt.

18. AAD (ASCII Adjust for Division)

Instrucțiunea modifică deîmpărțitul înaintea efectuării operației de împărțire a doi operanzi reprezentați sub forma de digiți necompactați.

Operații efectuate:

$AL \leftarrow AH \times 0AH + AL$

Indicatori afectați: PF, SF, ZF.

$AH \leftarrow 0$

19. CBW (Convert Byte to Word)

Instrucțiunea convertește un octet într-un cuvânt.

Operații efectuate:

if $AL < 80H$ then $AH \leftarrow 0$

else $AH \leftarrow 0FFH$

20. CWD (Convert Word to Doubleword)

Instrucțiunea convertește un cuvânt într-un dublu cuvânt.

Operații efectuate: if $AX < 8000H$ then $DX \leftarrow 0$

else $DX \leftarrow 0FFFFH$

Instrucțiunile de conversie sunt necesare pentru operațiile de împărțire, atunci când deîmpărțitul nu are formatul cerut de instrucțiune. De exemplu, dacă atât deîmpărțitul cât și împărțitorul se prezintă sub forma unui octet este necesară conversia deîmpărțitului într-un cuvânt înainte de efectuarea operației de împărțire.

INSTRUCTIUNI DE SALT

În această categorie sunt cuprinse instrucțiunile care determină modificarea secvenței normale, succesive, de execuție a instrucțiunilor. Ele acționează asupra regiștrilor CS și IP, întrucât aceștia stabilesc adresa următoarei instrucțiuni ce se va executa. Instrucțiunile de salt nu afectează indicatorii de condiții, unele fiind influențate de aceștia.

Există două tipuri de instrucțiuni de salt:

- în cadrul aceluiași segment în care se găsește instrucțiunea de salt (salt intrasegment); se modifică doar conținutul registrului IP;
- în cadrul altui segment (salt intersegment); se modifica conținutul regiștrilor CS și IP.

21. JMP OP (Jump)

OPERAND	SALT
imediat pe 8 biti	intrasegment
imediat pe 16 biti	intrasegment
imediat pe 32 biti	intersegment
adresa cuvânt	intrasegment
registru pe 16 biti	intrasegment
adresa dublu cuvânt	intersegment

Instrucțiunea execută un salt necondiționat la adresa indicată de operandul OP. Astfel, dacă OP apare ca un operand imediat, saltul se va face relativ la adresa instrucțiunii JMP. Dacă OP este un octet, saltul se va face în zona -126 până la +129 octeți, iar dacă este un cuvânt, saltul se va face în zona -32765 până la +32770 octeți față de adresa instrucțiunii JMP.

OP poate fi specificat ca un registru pe 16 biți sau ca adresa unei locații de memorie. În primul caz, conținutul registrului respectiv se încarcă în IP, iar în al doilea caz conținutul locației specificate și a următoareia se încarcă în IP, constituind deplasamentul adresei unde se execută saltul în cadrul segmentului de cod curent. Într-o altă situație există posibilitatea specificării unui registru pe 16 biți care va fi folosit ca registru indicator. Dacă OP este specificat ca un operand imediat pe dublu cuvânt, primul cuvânt se va încărca în IP, iar al doilea în CS, realizându-se un salt intersegment. Pentru a realiza un salt intersegment indirect se poate specifica OP ca adresa unei locații de memorie. Conținutul locației respective și a următoareia se va încărca în IP și conținutul următoarelor două locații se va încărca în CS.

JCC OP (Jump on Condition)

Îndrumar de laborator 6

Este o instrucțiune de salt condiționat. Dacă condiția CC este îndeplinită se execută saltul la adresa specificată de OP, iar în caz contrar instrucțiunea este ignorată. În cazul acestei instrucțiuni saltul nu se poate executa decât relativ la adresa instrucțiunii de salt și numai în zona -126 până la +129 octeți. Indicatorii testați sunt: CF, PF, SF, ZF și OF. Prin poziționarea indicatorilor CF și OF se pot compara operanzi cu și fără semn.

22. JA / JNBE OP (Jump on Above / Jump on Not Below or Equal)
Salt dacă mai mare / Salt dacă nu este mai mic sau egal (pentru operanzi fără semn).

Operație: if (CF & ZF) = 0 then IP ← IP + OP;
OP = operand imediat pe 8 biți.

23. JAE / JNB OP (Jump on Above or Equal / Jump on Not Below)
Salt dacă mai mare sau egal / Salt dacă nu este mai mic (pentru operanzi fără semn).

Operație: if CF = 0 then IP ← IP + OP;
OP = operand imediat pe 8 biți.

24. JB / JNAE OP (Jump on Below / Jump on Not Above or Equal)
Salt dacă mai mic / Salt dacă nu este mai mare sau egal (pentru operanzi fără semn).

Operație: if CF = 1 then IP ← IP + OP;
OP = operand imediat pe 8 biți.

25. JBE / JNA OP (Jump on Below or Equal / Jump on Not Above)
Salt dacă mai mic sau egal / Salt dacă nu este mai mare (pentru operanzi fără semn).

Operație: if (CF or ZF) = 1 then IP ← IP + OP;
OP = operand imediat pe 8 biți.

26. JC OP (Jump on Carry)
Salt dacă Carry = 1.

Operație: if CF = 1 then IP ← IP + OP;
OP = operand imediat pe 8 biți.

27. JE / JZ OP (Jump on Equal / Jump on Zero)
Salt dacă egal sau salt dacă zero.

Operație: if ZF = 1 then IP ← IP + OP;
OP = operand imediat pe 8 biți.

28. JG / JNLE OP (Jump on Greater / Jump on Not Less or Equal)

Îndrumar de laborator 6

Salt dacă mai mare / Salt dacă nu este mai mic sau egal (pentru operanzi cu semn).

Operație: if (SF=OF) or ZF=0 then $IP \leftarrow IP + OP$;
OP = operand imediat pe 8 biți.

29. JGE / JNL OP (Jump on Greater or Equal / Jump on Not Less)
Salt dacă mai mare sau egal / Salt dacă nu este mai mic (pentru operanzi cu semn).

Operație: if SF = OF then $IP \leftarrow IP + OP$;
OP = operand imediat pe 8 biți.

30. JLE / JNG OP (Jump on Less or Equal / Jump on Not Greater)
Salt dacă mai mic sau egal / Salt dacă nu este mai mare (pentru operanzi cu semn).

Operație: if (SF $\neq$ OF) or (ZF=1) then $IP \leftarrow IP + OP$;
OP = operand imediat pe 8 biți.

31. JNC OP (Jump on Not Carry)

Salt dacă CF = "0" logic.

Operație: if CF = 0 then $IP \leftarrow IP + OP$;
OP = operand imediat pe 8 biți.

32. JNE / JNZ OP (Jump on Not Equal / Jump on Not Zero)

Salt dacă nu este egal (ZF="0" logic).

Operație: if ZF = 0 then $IP \leftarrow IP + OP$;
OP= imediat pe 8 biți.

33. JNO OP (Jump on Not Overflow)

Salt dacă nu este depășire (OF="0" logic).

Operație: if OF = 0 then $IP \leftarrow IP + OP$;
OP= imediat pe 8 biți.

34. JNP / JPO OP (Jump on Not Parity / Jump on Parity Odd)

Salt dacă nu este paritate / Salt la paritate impară.

Operație: if PF = 0 then $IP \leftarrow IP + OP$;
OP = operand imediat pe 8 biți.

35. JNS OP (Jump on Not Sign)

Salt dacă SF = "0" logic.

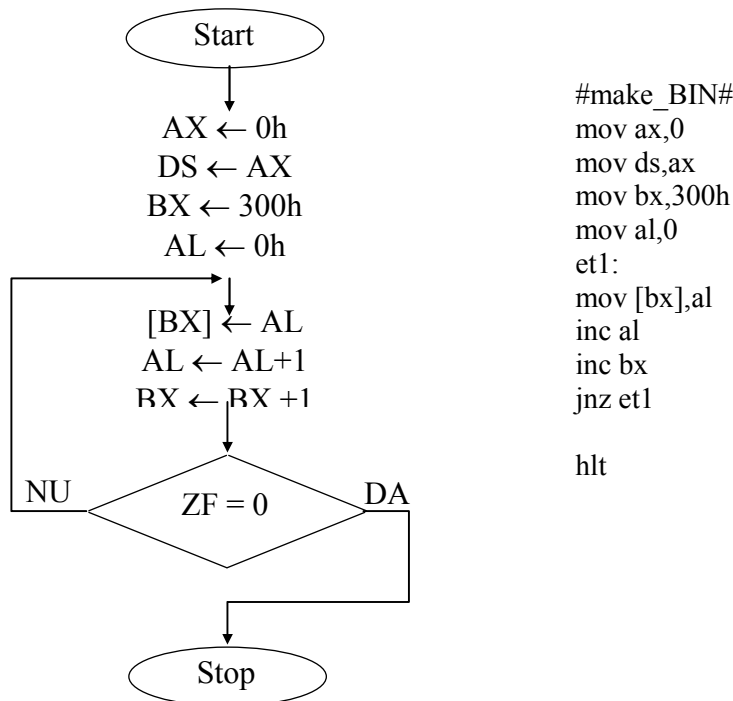
Operație: if SF = 0 then $IP \leftarrow IP + OP$;
OP = operand imediat pe 8 biți

Îndrumar de laborator 6

36. JO OP (Jump on Overflow)
Salt dacă este depășire (OF = "1" logic).
Operație: if OF = 1 then IP←IP + OP;
OP = operand imediat pe 8 biți.
37. JP / JPE OP (Jump on Parity / Jump on Parity Equal)
Salt la paritate / Salt la paritate pară.
Operație: if PF = 1 then IP←IP + OP;
OP = operand imediat pe 8 biți.
38. JS OP (Jump on Sign)
Salt dacă SF = "1" logic.
Operație: if SF = 1 then IP←IP + OP;
OP = operand imediat pe 8 biți.
39. JCXZ OP (Jump if CX register Zero)
Instrucțiunea execută un salt dacă conținutul registrului CX este zero. OP poate fi specificat doar în forma imediată pe un octet ceea ce înseamnă ca saltul nu se poate efectua decât relativ la adresa instrucțiunii de salt și numai în zona -126 până la +129 octeți. Instrucțiunea este utilă atunci când se folosesc bucle. Cum CX indică numărul de treceri prin bucla instrucțiunea permite ieșirea dintr-o bucla sau ocolirea ei.
Operație: if CX=0 then IP←IP+OP.
OP = operand imediat pe 8 biți.

Probleme rezolvate

1. Să se realizeze un program, prin intermediul căruia să se depună în memoria RAM a unui microsistem cu microprocesor 8086, începând de la adresa 300h, un șir de numere continuu ce conține date de la 00h până la 0ffh.



Microprocesorul 8086 dispune de un mecanism de adresare segmentată a memoriei. Pentru a se realiza adresarea locației de memorie 300h, este necesar să se inițializeze doi regiștri și anume:

- **DS** – registrul segment, încărcat cu valoarea 0h;
- **BX** – offsetul, încărcat cu valoarea 300h.

Calculul adresei fizice a locației de memorie se face astfel:

- se deplasează valoarea aflată în registrul DS la stânga cu 4 poziții și se introduce pe locurile rămase libere zero;
- se adună valoarea rezultată mai sus cu valoarea aflată în registrul BX, rezultând o valoare pe 20 de biți, valoare care reprezintă adresa fizică a locației de memorie adresată prin intermediul registrului de segment (DS) și offset (BX).

Îndrumar de laborator 6

Pentru a depune la locații de memorie consecutive șiruri de numere, se realizează o buclă, în interiorul căreia se incrementează registrul BX (offsetul care adresează memoria) și registrul AL (care reprezintă data care se depune). În interiorul buclei se rămâne cât timp indicatorul ZF este diferit de 1. când registrul AL atinge valoarea 0h, ZF devine 1 și se trece la executarea instrucțiunii următoare.

Pentru a rula programul și a observa dacă se execută corect se va folosi programul emulator. După editarea programului acesta se compilează și emulează. Pentru a vizualiza zona de memorie unde au fost depuse numerele se va ține cont că această zonă se află la adresa 0000:0300 (segment:offset).

De rezolvat:

- a. Realizați un program care să depună la adresa 300h la un microsistem cu microprocesor 8086, un șir de numere continuu ce conține date de la 0ffh la 00h, în ordine descrescătoare.
- b. Realizați un program care să depună la adresa 600h la același microsistem, un șir crescător de 50 de numere pare (ex. 0, 2, 4,...).
- c. Realizați un program care să depună în memoria RAM al aceluiași microsistem, începând cu adresa 600h un șir continuu de 100 de numere astfel: primele 50 de numere vor fi depuse la adrese pare (ex. 600h, 602h, ...) iar următoarele 50 de numere la adrese impare (ex. 601h, 603h, ...).

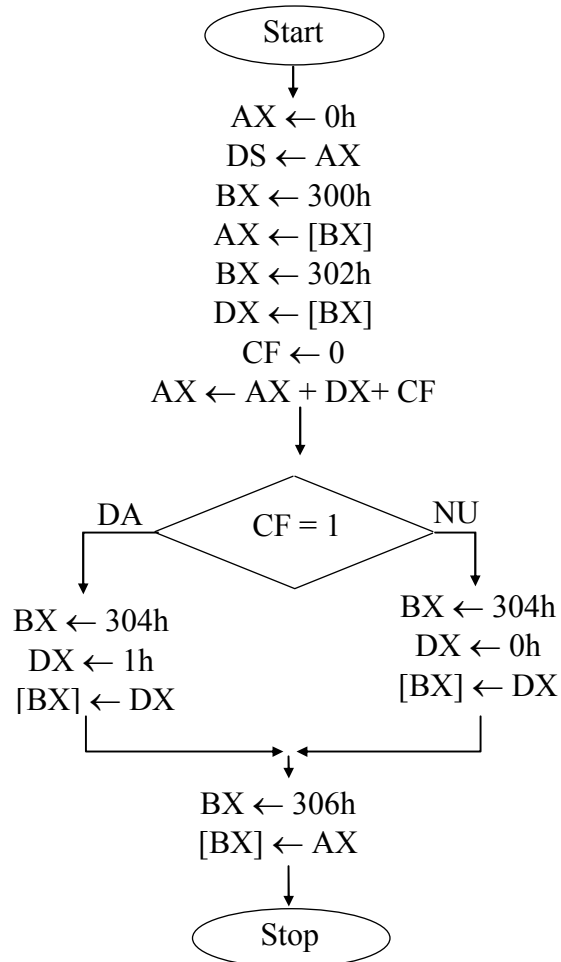
Îndrumar de laborator 6

2. Să se realizeze un program prin intermediul căruia să se efectueze suma a două numere pe 16 biți aflate la adresele 300h și 302h, iar rezultatul să se depună la adresa 304h.

```
#make_BIN#

mov ax,0
mov ds,ax
mov bx,300h
mov ax,[bx]
mov bx,302h
mov dx,[bx]
clc
adc ax,dx

jc et1
mov bx,304h
mov dx,0
mov [bx],dx
et2: mov bx,306h
     mov [bx],ax
     jmp et3
et1:  mov bx,304h
     mov dx,1
     mov [bx],dx
     jmp et2
et3:  hlt
```



Operanzii care urmează a fi supuși operației de adunare sunt gestionați în cadrul registrului segment **DS**. Valoarea inițială a acestuia este 0h. Adresa primului operand este specificată prin intermediul registrului **BX** și acesta este transferat în cadrul registrului **AX**. Cel de-al doilea operand este transferat în memorie în cadrul registrului **DX**. Operația de adunare efectuată este de tipul „cu carry”, acest indicator fiind inițial încărcat cu valoarea 0. rezultatul operației propuse prin enunț, în condițiile unor operanzi maximi, se reprezintă pe 3 octeți. În cadrul rezolvării

Îndrumar de laborator 6

propunem o reprezentare a rezultatului pe 4 octeți. La plasarea acestuia în memorie se ține seama de existența sau nu a transportului. Dacă acesta nu există ($CY = 0$), rezultatul, pe un singur cuvânt, se depune în doi octeți succesivi în memorie, iar următorii doi (mai semnificativi) sunt încărcăți cu valoarea 0. Altfel ($CY = 1$) la rezultat se adaugă încă doi octeți în care se depune valoarea indicatorului carry (pe bitul cel mai puțin semnificativ). Pentru a vedea un rezultat prin simulare, la adresele 300h și 302h (segment 0) trebuie introduși cei doi operanzi.

De rezolvat:

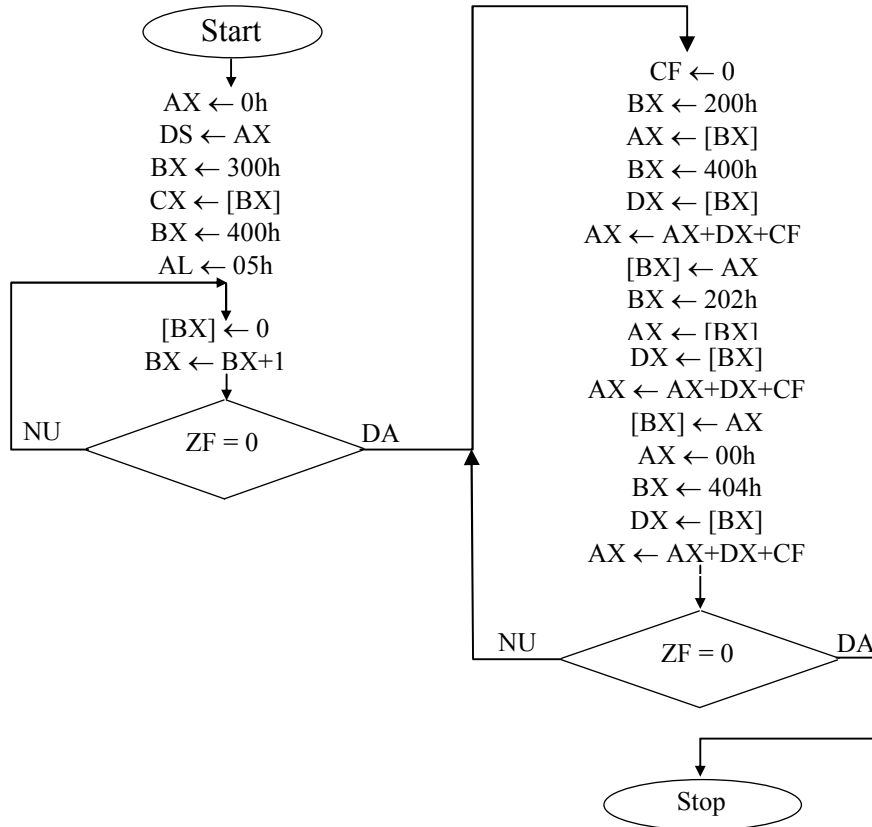
a. Să se realizeze un program prin intermediul căruia să se efectueze scăderea a două numere pe 16 biți aflate la adresele 300h și 302h, iar rezultatul să se depună la adresa 304h.

b. Să se realizeze un program care să efectueze adunarea a patru numere pe 16 biți aflate la adresele 300h, 302h, 304h și 306h. rezultatul va fi depus începând cu adresa 310h.

3. În memoria unui microsistem cu microprocesor 8086 se află plasați doi operanzi, primul reprezentat pe 4 octeți începând cu adresa 200h și cu octetul cel mai puțin semnificativ, al doilea reprezentat pe doi octeți începând cu adresa 300h și cu octetul cel mai puțin semnificativ. Se cere realizarea unui program care efectuează operația de înmulțire a celor două numere cu plasarea rezultatului începând cu adresa 400h și cu octetul cel mai puțin semnificativ.

- a. Să se realizeze înmulțirea a 2 numere, reprezentate pe 2 octeți fiecare. Cele două numere sunt plasate în memorie la adresele 200h respectiv 210h. Rezultatul va fi plasat la adresa 220h.
- b. Să se realizeze împărțirea a două numere unul pe 4 octeți (deîmpărțitul) iar celălalt pe 2 octeți (împărțitorul). Numere sunt plasate la adresele 200h respectiv 210h. Rezultatul va si pus la adresa 220h

Îndrumar de laborator 6



#make_BIN#

```

mov ax,0
mov ds,ax
mov bx,200h
mov cx,[bx]
mov bx,300h
mov al,05h

et1:
  mov [bx],0
  inc bx
  dec al
  jnz et1

et2:
  cld
  mov bx,100h
  mov ax,[bx]
  mov bx,300h
  
```

```

mov dx,[bx]
adc ax,dx
mov [bx],ax
mov bx,102h
mov ax,[bx]
mov bx,302h
mov dx,[bx]
adc ax,dx
mov [bx],ax
mov bx,104h
mov ax,[bx]
mov bx,304h
mov dx,[bx]
adc ax,dx
dec cx
jnz et2

hl
  
```

Îndrumar de laborator 6